

Vector Space Model and Ranked Retrieval on the Cranfield Collection

Information Retrieval — Programming Assignment II

1. Group details

Group name: **search_ninjas**. All submitted files carry this prefix.

- Krish Charniya

2. Problem statement

Build a ranked retrieval pipeline over the Cranfield collection (1,400 aerodynamics abstracts, 225 test queries, 1,612 graded relevance judgements) using an open-source search engine, and tune the preprocessing, the weighting model and its parameters to maximise retrieval effectiveness. Only sparse lexical models are permitted — no dense or neural retrieval. Indexing and search times are to be reported alongside effectiveness, and the resulting system is to be evaluated on further unseen queries.

3. Implementation details

The system is built on **PyTerrier 1.1.2** driving **Terrier 5.11**. Terrier supplies the inverted index, the term pipeline and the weighting models; our code supplies the collection parsing, the experiment plan and the evaluation harness.

3.1 Programs

File	Role
search_ninjas_setup.sh	creates the virtualenv, fetches a JDK, unpacks the collection
search_ninjas_env.py	points JAVA_HOME at the bundled JDK before pyjnius loads the JVM
search_ninjas_parse.py	readers for cran.all.1400, cran.qry and cranqrel
search_ninjas_index.py	builds a Terrier index under one preprocessing variant, timed
search_ninjas_experiments.py	the four-stage experiment plan; writes every table in this report
search_ninjas_search.py	runs the tuned pipeline over a query file, writes a TREC run
search_ninjas_report.py	renders this PDF from the experiment output

3.2 Preprocessing

The PA1 preprocessing steps are repeated, but as Terrier's term pipeline rather than our own code: case folding and non-alphanumeric stripping in the English tokeniser, stopword removal against the list supplied with PA1, and Porter stemming. Each step is switchable so that Stage 1 can measure what it contributes. The same normalisation is applied to query text before it reaches Terrier's parser, so query terms and index terms cannot drift apart.

3.3 Two properties of the collection that the code has to get right

Query numbering. `cranqrel` numbers the queries 1-225 by their position in `cran.qry`, whereas `cran.qry`'s own `.I` labels are non-contiguous (001, 002, 004, 008, ... 365). Keying a run on the labels and scoring it against `cranqrel` silently evaluates almost every query against the wrong judgements. Our run files are written under both numberings.

Relevance grades. Cleverdon's codes run 1 (a complete answer) to 4 (minimum interest) — smaller is better, the reverse of what every evaluation tool assumes. They are inverted to gains 4-1 before use; the 225 rows coded -1 are unjudged markers and are dropped.

Empty records. Documents 471 and 995 have no title, author, bibliography or abstract at all. Document 995 is judged relevant for query 125, so it is an unreachable relevant document that places a hard ceiling on recall.

3.4 Parsing

A Cranfield record is written `.T .A .B .W`. The document parser is a state machine keyed on the most recent field tag with one extra rule: once `.W` has opened, an `.A` or `.B` line is stray text inside the abstract rather than a new field. Document 240 contains exactly this, and without the rule it loses roughly fifteen lines of its abstract.

4. Plan of experiments

Four stages, each one narrowing the next: **(1)** preprocessing — which fields, stemmer and stopword list to index; **(2)** weighting model at default parameters; **(3)** the parameters of every parametrised model; **(4)** pseudo-relevance feedback on top of the winner.

Stages 3 and 4 tune on the **odd-numbered** queries (113 of them) and report on the held-out **even-numbered** queries (112), as well as on the full set. The interleaved split is deliberate: the Cranfield queries are grouped by subject, so a contiguous split would tune on a different subject mix than it reports on. Every figure labelled held-out below comes from queries the tuning never saw, which is the honest estimate of what the unseen test queries will produce.

Effectiveness is measured with `ir_measures`: MAP (the primary metric used for all selection), `nDCG@10` and `nDCG@20` on the graded judgements, `P@5`, `P@10`, `recall@100` and reciprocal rank. Improvements over a baseline are accompanied by a paired sign test; the + and - columns count the queries that improved and degraded.

5. Results

5.1 Stage 1 — preprocessing

Twenty-seven indices: three field sets × three stemmers × three stopword settings, each scored with untuned BM25.

Configuration	AP	nDCG@10	P@10	R@100	terms	postings	avg len	index (s)	search (ms/q)
all_porter_assignment	0.3185	0.3763	0.2382	0.7376	6,495	92,349	106.24	0.41	7.22
title_text_porter_assignment	0.3156	0.3729	0.2378	0.7397	4,603	80,630	97.36	0.35	7.83
all_porter_terrier	0.3132	0.3706	0.2396	0.7426	6,452	89,178	103.80	0.32	7.34
title_text_porter_terrier	0.3116	0.3680	0.2391	0.7446	4,561	81,049	97.86	0.32	8.15
all_weakporter_assignment	0.3021	0.3586	0.2360	0.7303	7,518	94,232	106.24	0.34	7.54
text_porter_terrier	0.3006	0.3575	0.2347	0.7445	4,561	81,049	89.88	0.33	7.56
text_porter_assignment	0.3005	0.3602	0.2342	0.7366	4,603	80,630	89.43	0.37	7.47
title_text_weakporter_assignment	0.2995	0.3552	0.2351	0.7302	5,601	82,512	97.36	0.40	7.21
all_weakporter_terrier	0.2978	0.3518	0.2329	0.7371	7,465	91,036	103.80	0.37	6.97
title_text_weakporter_terrier	0.2963	0.3504	0.2329	0.7383	5,549	82,906	97.86	0.63	8.55
all_none_assignment	0.2904	0.3538	0.2324	0.7101	9,128	98,273	106.24	0.32	6.15
all_none_terrier	0.2895	0.3538	0.2342	0.7112	9,085	95,066	103.80	0.35	6.30
title_text_none_terrier	0.2887	0.3527	0.2329	0.7122	7,147	86,930	97.86	0.34	7.07
title_text_none_assignment	0.2887	0.3526	0.2302	0.7124	7,189	86,548	97.36	0.56	8.34
text_weakporter_assignment	0.2882	0.3455	0.2329	0.7262	5,601	82,512	89.43	0.30	7.09
text_weakporter_terrier	0.2854	0.3400	0.2298	0.7358	5,549	82,906	89.88	0.38	6.82
text_none_assignment	0.2796	0.3439	0.2262	0.7088	7,189	86,547	89.43	0.36	6.12
text_none_terrier	0.2772	0.3405	0.2271	0.7106	7,147	86,929	89.88	0.60	6.80
title_text_porter_none	0.2079	0.2414	0.1591	0.6640	4,815	116,502	173.81	0.36	9.52
all_porter_none	0.2054	0.2424	0.1613	0.6600	6,705	128,354	183.46	0.35	8.73
all_weakporter_none	0.2010	0.2374	0.1613	0.6471	7,739	130,514	183.46	0.36	9.20
title_text_weakporter_none	0.2003	0.2337	0.1582	0.6514	5,822	118,643	173.81	0.43	10.78
all_none_none	0.1945	0.2300	0.1529	0.6215	9,400	134,796	183.46	0.48	9.32
title_text_none_none	0.1927	0.2283	0.1556	0.6224	7,458	122,918	173.81	0.35	9.17
text_porter_none	0.1926	0.2243	0.1511	0.6494	4,815	116,502	161.90	0.38	9.16
text_weakporter_none	0.1864	0.2154	0.1471	0.6423	5,822	118,643	161.90	0.36	9.16
text_none_none	0.1795	0.2110	0.1440	0.6121	7,458	122,917	161.90	1.37	13.19

Table 1: all 27 preprocessing variants, ordered by MAP. Names read *fields__stemmer__stopwords*.

5.2 Stage 2 — weighting models

All models at their default parameters, on the Stage 1 winning index.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	search (ms/q)
ln_expB2	0.3290	0.3896	0.4291	0.3342	0.2538	0.7571	0.5543	6.86
TF_IDF	0.3203	0.3778	0.4197	0.3307	0.2373	0.7470	0.5569	7.62
BM25	0.3185	0.3763	0.4192	0.3316	0.2382	0.7376	0.5498	6.77
DFR_BM25	0.3184	0.3772	0.4201	0.3307	0.2382	0.7382	0.5528	6.74
PL2	0.3136	0.3734	0.4175	0.3244	0.2347	0.7438	0.5591	6.78
IFB2	0.3128	0.3724	0.4101	0.3200	0.2431	0.7256	0.5322	7.14
DPH	0.3127	0.3777	0.4167	0.3209	0.2329	0.7395	0.5749	7.11
DLH13	0.3093	0.3674	0.4118	0.3209	0.2267	0.7337	0.5569	6.90
DFree	0.3032	0.3603	0.4011	0.3138	0.2187	0.7255	0.5560	7.03
Hiemstra_LM	0.2984	0.3544	0.3993	0.3093	0.2173	0.7346	0.5360	7.10
LemurTF_IDF	0.2953	0.3543	0.3914	0.2969	0.2284	0.7265	0.5045	7.34
Tf	0.1927	0.2352	0.2767	0.1964	0.1507	0.6516	0.4302	8.70

Table 2: weighting models at default parameters over the full 225 queries.

5.3 Stage 3 — parameter tuning

110 BM25 settings ($k_1 \times b$) and 44 DFR settings (c), all scored on the training half. The best of each family:

Model	Configuration	AP	nDCG@10	P@10	R@100
ln_expB2	ln_expB2(c=0.5)	0.3409	0.3914	0.2664	0.7796
BM25	BM25($k_1=2.5$, $b=0.75$)	0.3383	0.3905	0.2540	0.7673
DFR_BM25	DFR_BM25(c=0.5)	0.3361	0.3867	0.2496	0.7633
PL2	PL2(c=1.0)	0.3309	0.3841	0.2451	0.7596
IFB2	IFB2(c=1.0)	0.3230	0.3771	0.2460	0.7422

Table 3: best parameter setting per model family, training queries only.

Tuned against default for the winning model, on the held-out half:

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
ln_expB2(c=1.0) [default]	0.3177	0.3879	0.4192	0.3321	0.2491	0.7461	0.5545			
ln_expB2(c=0.5) [tuned]	0.3230	0.3949	0.4281	0.3393	0.2500	0.7561	0.5630	64	38	0.190600

Table 4: effect of parameter tuning on the 112 held-out queries.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
ln_expB2(c=1.0) [default]	0.3290	0.3896	0.4291	0.3342	0.2538	0.7571	0.5543			
ln_expB2(c=0.5) [tuned]	0.3320	0.3931	0.4324	0.3440	0.2582	0.7679	0.5494	125	79	0.368856

Table 5: the same comparison over all 225 queries.

5.4 Stage 4 — pseudo-relevance feedback

Bo1 and KL query expansion, three feedback-document counts $\times$ three feedback-term counts, on the training half:

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR
Bo1(docs=5, terms=20)	0.3740	0.4152	0.4570	0.3575	0.2805	0.7922	0.5419
KL(docs=5, terms=20)	0.3705	0.4102	0.4542	0.3575	0.2770	0.7945	0.5418
Bo1(docs=5, terms=5)	0.3679	0.4095	0.4530	0.3558	0.2770	0.7912	0.5376
KL(docs=5, terms=10)	0.3674	0.4080	0.4504	0.3540	0.2752	0.7923	0.5430
Bo1(docs=5, terms=10)	0.3669	0.4080	0.4526	0.3593	0.2761	0.7944	0.5335
KL(docs=5, terms=5)	0.3645	0.4071	0.4504	0.3540	0.2752	0.7913	0.5412
Bo1(docs=3, terms=20)	0.3638	0.4114	0.4510	0.3628	0.2770	0.7904	0.5542
Bo1(docs=3, terms=10)	0.3634	0.4126	0.4510	0.3593	0.2761	0.7845	0.5616
Bo1(docs=10, terms=20)	0.3615	0.4080	0.4494	0.3451	0.2796	0.7975	0.5492

Table 6: the nine best of eighteen expansion settings, training queries only.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
In_expB2(c=0.5)	0.3230	0.3949	0.4281	0.3393	0.2500	0.7561	0.5630			
In_expB2(c=0.5) + Bo1(docs=5, terms=20)	0.3456	0.4095	0.4485	0.3589	0.2723	0.7945	0.5459	71	39	0.006188

Table 7: expansion on the 112 held-out queries.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
In_expB2(c=0.5)	0.3320	0.3931	0.4324	0.3440	0.2582	0.7679	0.5494			
In_expB2(c=0.5) + Bo1(docs=5, terms=20)	0.3599	0.4124	0.4527	0.3582	0.2764	0.7933	0.5439	139	74	0.000009

Table 8: expansion over all 225 queries.

5.5 Final configuration and timings

Setting	Value
Indexed fields	all
Stemmer	porter
Stopword list	assignment
Weighting model	In_expB2 (c = 0.5)
Query expansion	Bo1(docs=5, terms=20)
Indexing time (1,400 documents)	1.19 s
Search time, no expansion	6.7 ms / query
Search time, with expansion	22.1 ms / query
MAP, held-out queries	0.3456
MAP, all 225 queries	0.3599

Table 9: the submitted configuration. Indexing is a single-threaded in-memory build; search time is wall clock over all 225 queries divided by 225.

6. Discussion of results

6.1 Preprocessing dominates everything else

The single largest effect in the whole study is stopword removal. The best variant with no stopword list reaches MAP 0.2079; the best with one reaches 0.3185 — a gain of +53.2%, larger than every model and parameter decision combined. Cranfield queries are full English questions ("what similarity laws must be obeyed when..."), so without a stopword list the query vector is dominated by terms that match almost every document. Porter stemming is the second largest effect, worth roughly 0.03 MAP, and switching from Porter to Terrier's weak Porter loses most of that — on 1,400 short abstracts, aggressive conflation is the right trade.

Field choice barely matters. Adding the author and bibliography fields to the title and abstract is worth about 0.003 MAP, and indexing the abstract alone rather than title plus abstract costs about 0.015 — small because in most Cranfield records the title is repeated as the opening of the abstract, so the title field is nearly redundant already.

6.2 Model choice matters more than model parameters

Raw term frequency with no IDF component (MAP 0.1927) is less than two thirds as effective as anything that weights by term rarity, which is the expected confirmation that the vector space model's IDF component is doing the work. Among the properly weighted models the spread is narrow — TF-IDF, BM25, DFR-BM25, PL2 and DPH all land between 0.31 and 0.33 MAP — with `In_expB2` slightly ahead.

Parameter tuning then adds remarkably little. On the held-out queries the tuned model gains +0.0053 MAP over its own default, improving 64 queries and degrading 38 for a sign-test p-value of 0.191 — not significant. The BM25 response surface is nearly flat across the whole grid: every setting between $k_1 = 0.8$ and 4.0 and $b = 0.3$ and 1.0 scores within about 0.01 MAP. This is worth stating plainly rather than reporting the tuned number alone, because the difference between the best and the default setting here is smaller than the noise between two random halves of the query set.

One methodological note: an earlier, narrower sweep put the optimal k_1 exactly on the boundary of the grid, which is a sign that the grid is too small rather than an answer. The grid reported above extends to $k_1 = 4.0$ so that the optimum is interior.

6.3 Query expansion is the one intervention that clearly pays

Bo1 pseudo-relevance feedback improves MAP from 0.3230 to 0.3456 on the held-out queries, +7.0%, improving 71 queries against 39 degraded — sign-test $p = 0.006188$. Unlike the parameter tuning, this survives on queries the tuning never saw, and it is the only change in the study that does so convincingly. It is also the only change with a real cost: expansion runs the retrieval twice and roughly triples search time, from 6.7 ms to 22.1 ms per query. At Cranfield's scale that is irrelevant; on a large collection it would be the deciding factor.

Why it helps so much here is specific to the collection: Cranfield abstracts are short and the vocabulary is narrow and technical, so the top few documents for a query are densely packed with exactly the domain terms the query failed to mention. Feedback from five documents is best; ten begins to drift, three is too thin a sample.

6.4 Where the ceiling is

End to end, the pipeline moves MAP from 0.3185 (untuned BM25 on the winning index) to 0.3599 over the full query set, +13.0%. Recall@100 finishes at 0.7933, so roughly a fifth of the relevant documents are still missed in the top 100 — and at least one of them, document 995 for query 125, is unreachable by any lexical system because the record is empty. The remaining gap is mostly vocabulary mismatch that term expansion from the top five documents cannot bridge, which is precisely the failure mode that dense retrieval addresses and that this assignment excludes.

6.5 Threats to validity

The held-out half is 112 queries, so a MAP difference below roughly 0.01 is not distinguishable from noise; we have reported sign tests rather than relying on the point estimates. Stage 1 selected the index on the full query set before the train/test split was introduced in Stages 3 and 4, so the preprocessing choice is mildly optimistic — though with an effect that large, and a ranking that is stable across every model we tried, the conclusion is not in doubt.